

SIMATS ENGINEERING

ASSESSMENT TOOL 2

COURSE CODE: CSA51 COURSE NAME: Cryptography and Network Security

COURSE OUTCOME COVERED

CO4: *Implement best security practices for IP, email, and web service using PGP, S/MIME for enhancing email Security.CO (BL3, BL6)*

Assessment Tool Weightage: 20%

Code of Conduct:

I C. Sangita(192373059) (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.

I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student:

Sangita.c

Annexure A

1. SSL/TLS Overhead and Web Server Performance Optimization

A web server uses TLS 1.3 for secure communication. Each HTTP response carries 2000 bytes of payload. TLS adds 25 bytes (record header + authentication tag) and handshake overhead of 3 KB per session. The server handles 10,000 requests per second, with 1,000 new TLS handshakes per second.

Task:

- A. Calculate the total data transmitted per second including TLS overhead.
- B. Determine the percentage overhead due to TLS encryption.
- C. Estimate bandwidth consumed due to handshake overhead per second.
- D. Analyze the trade-off between security and latency, and propose optimization techniques (e.g., session resumption, HTTP/2, or QUIC).

5. Email Encryption Latency in Cloud-Based Systems

A cloud email service encrypts outgoing emails using AES (data) and RSA (key exchange).

AES encryption takes 0.2 ms per MB, and RSA key exchange takes 5 ms per email. Each email has 8 MB attachment. The system processes 8,000 emails per hour.

Task:

- A. Calculate total encryption time per email.
- B. Estimate total processing time per hour.
- C. Analyze the performance bottleneck between symmetric and asymmetric operations.
- D. Recommend optimization strategies (e.g., ECC adoption, parallel processing, or hardware acceleration).

ANSWERS:

QUESTION 1

SSL/TLS Overhead and Web Server Performance Optimization

Problem Statement

A web server uses TLS 1.3 for secure communication. Each HTTP response carries 2000 bytes of payload. TLS adds 25 bytes of record overhead. The handshake overhead is 3 KB per session. The server handles 10,000 requests per second, with 1,000 new TLS handshakes per second.

Given Data

- HTTP payload = 2000 bytes
- TLS overhead = 25 bytes
- Requests = 10,000 requests/second
- New TLS handshakes = 1,000/second
- Handshake overhead = 3 KB/session
- 1 KB = 1024 bytes

A. Total Data Transmitted Per Second

Size of one response including TLS overhead:

For 10,000 requests:

Therefore:

In bits:

Thus, the web server transmits approximately 20.25 MB/s or 162 Mbps of application data including TLS record overhead.

B. Percentage Overhead Due to TLS

TLS adds 25 bytes to a 2000-byte payload.

Therefore:

The TLS record overhead is only 1.25%, which is relatively small compared with the original data.

For 10,000 requests per second, the record overhead is:

Thus, TLS record headers and authentication tags add approximately:

C. Bandwidth Consumed by Handshake Overhead

Each handshake requires 3 KB.

There are 1,000 handshakes per second.

Therefore:

In Mbps:

Therefore, the TLS handshake consumes approximately:
of bandwidth.

Total Estimated Bandwidth

Application traffic:

Handshake traffic:

Therefore:

In Mbps:

D. Security and Latency Trade-Off

TLS 1.3 provides three major security benefits:

1. Confidentiality

TLS encrypts information so attackers cannot read sensitive data such as passwords, payment information and personal details.

2. Integrity

TLS helps detect unauthorized modification of transmitted data.

3. Authentication

Digital certificates allow the client to verify the identity of the web server.

However, TLS introduces some overhead. A new handshake requires communication between the client and server and cryptographic processing. If too many new connections are created, latency and CPU usage increase.

In this problem, the record overhead is only 1.25%, but the 1,000 new handshakes per second produce 3.072 MB/s of additional traffic.

Therefore, reducing unnecessary handshakes is important for performance.

Optimization Techniques

1. TLS Session Resumption

Session resumption allows a client to reconnect without performing a complete TLS handshake.

Benefits:

- Reduces latency.

- Reduces bandwidth.
- Reduces CPU usage.
- Improves server performance.

2. Persistent Connections

Multiple HTTP requests can use the same secure connection. This avoids repeatedly creating new TLS sessions.

3. HTTP/2

HTTP/2 supports multiplexing, allowing multiple requests to share a single connection. This reduces connection overhead and improves performance.

4. HTTP/3 and QUIC

HTTP/3 uses QUIC and TLS 1.3. It provides efficient connection establishment and better performance under changing network conditions.

5. Hardware Acceleration

Cryptographic hardware or CPU acceleration can reduce the processing time required for encryption and authentication.

6. Load Balancing

TLS processing can be distributed among multiple servers. This prevents a single server from becoming overloaded.

Industry Application

The same problem occurs in:

- Banking websites
- E-commerce websites
- Cloud applications
- Online education platforms
- Payment gateways
- Government portals
- REST APIs

These services cannot remove TLS because sensitive information must remain protected.

Therefore, the goal is to reduce unnecessary TLS overhead while maintaining strong security.

Parameter	Result
Total response size	2025 bytes
Data per second	20.25 MB/s
TLS record overhead	1.25%
Handshake bandwidth	3.072 MB/s
Handshake bandwidth	24.576 Mbps
Total estimated bandwidth	23.322 MB/s
Total estimated bandwidth	186.576 Mbps

Conclusion

TLS 1.3 provides strong confidentiality, integrity and authentication. The record overhead is only **1.25%**, but frequent handshakes create significant additional bandwidth and latency. Session resumption, persistent connections, HTTP/2, HTTP/3/QUIC, hardware acceleration and load balancing can improve performance without sacrificing security.

QUESTION 5

Email Encryption Latency in Cloud-Based Systems

Problem Statement

A cloud email service encrypts outgoing emails using AES for data encryption and RSA for key exchange. AES encryption takes 0.2 ms per MB, and RSA key exchange takes 5 ms per email. Each email has an 8 MB attachment. The system processes 8,000 emails per hour.

Given Data

- AES encryption = 0.2 ms/MB
- Attachment = 8 MB
- RSA key exchange = 5 ms/email
- Emails = 8,000/hour

A. Total Encryption Time Per Email

AES Encryption

AES takes:

For an 8 MB attachment:

Therefore:

RSA Key Exchange

Given:

Total Time

Therefore, each email requires 6.6 ms of cryptographic processing.

B. Total Processing Time Per Hour

The system processes:

Each requires:

Therefore:

Converting to seconds:

Therefore, the cumulative cryptographic processing time is:

if all emails are processed sequentially.

In a real cloud system, emails can be processed simultaneously using multiple workers, so the actual elapsed time can be much lower.

C. Performance Bottleneck Analysis

AES takes:

RSA takes:

Total:

RSA Percentage

AES Percentage

Therefore:

RSA is therefore the main performance bottleneck.

Why AES and RSA Are Used Together

AES is a symmetric encryption algorithm and is very fast for large amounts of data.

RSA is an asymmetric algorithm and is slower, so it should not normally be used to encrypt an entire 8 MB attachment.

A hybrid encryption system works as follows:

1. Generate a random AES session key.
2. Encrypt the email attachment using AES.
3. Encrypt/protect the AES session key using RSA.
4. Send the encrypted attachment and protected session key.

5. The receiver uses the RSA private key to recover the AES key.
6. The receiver uses AES to decrypt the attachment.

This approach provides both security and efficiency.

D. Optimization Strategies

1. ECC Adoption

ECC, or Elliptic Curve Cryptography, can provide strong security using smaller key sizes than traditional RSA.

Possible benefits include:

- Lower computational requirements.
- Smaller keys.
- Reduced communication overhead.
- Efficient key establishment.

ECC can therefore be considered for modern cloud systems where the required email protocols support it.

2. Parallel Processing

Instead of processing one email at a time, multiple emails can be encrypted simultaneously.

For example:

Worker 1 → Email 1

Worker 2 → Email 2

Worker 3 → Email 3

Worker 4 → Email 4

This increases throughput and makes better use of multiple CPU cores.

3. Hardware Acceleration

Modern processors provide hardware support for many cryptographic operations.

Hardware acceleration can:

- Reduce CPU usage.
- Reduce encryption latency.
- Increase throughput.
- Improve energy efficiency.

This is particularly useful for large cloud email systems.

4. Cloud Autoscaling

During peak periods, the cloud service can automatically create additional encryption workers.

For example:

During low traffic:

This improves resource utilization and reduces operating costs.

5. Efficient Key Management

The organization should implement:

- Secure key generation.
- Key rotation.
- Private-key protection.
- Access control.
- Key revocation.
- Secure key storage.

Proper key management is essential because encryption is ineffective if the cryptographic keys are compromised.

Industry Application

Cloud-based email encryption is important for:

- Banks
- Hospitals
- Government organizations
- Universities
- IT companies
- Financial institutions
- Legal organizations
- E-commerce companies

These organizations frequently exchange confidential documents through email.

A secure email system must provide confidentiality while also processing a large number of messages quickly.

Security Considerations

Confidentiality

Encryption prevents unauthorized users from reading email attachments.

Authentication

Cryptographic authentication can verify the identity of communicating parties.

Integrity

Integrity mechanisms can detect unauthorized modifications.

Key Protection

Private cryptographic keys must be protected against unauthorized access.

Key Rotation

Keys should be periodically replaced according to organizational security policies.

Performance Comparison

Operation	Time per Email	Percentage
AES encryption	1.6 ms	24.24%
RSA key exchange	5 ms	75.76%
Total	6.6 ms	100%

The table clearly shows that RSA is responsible for the larger part of the cryptographic processing time.

A. Total Encryption Time

AES:

RSA:

Total:

B. Total Processing Time

C. Bottleneck

RSA:

AES:

Therefore, RSA is the major bottleneck.

D. Recommendations

1. Adopt ECC-based key establishment where suitable.

2. Use parallel processing.
3. Use hardware cryptographic acceleration.
4. Continue using hybrid AES + asymmetric encryption.
5. Implement cloud autoscaling.
6. Improve key-management processes.

Conclusion

The cloud email system requires 6.6 ms of cryptographic processing per email. For 8,000 emails, the cumulative processing requirement is 52.8 seconds per hour when calculated sequentially.

AES is efficient for encrypting the 8 MB attachment and contributes only 24.24% of the cryptographic time. RSA contributes 75.76%, making it the major bottleneck.

Therefore, the system should use hybrid encryption, with AES handling large data and asymmetric cryptography handling key protection. ECC, parallel processing, hardware acceleration, autoscaling, and efficient key management can improve the overall performance while maintaining strong security.